

Build Brief: Legal Practice Management PWA

You are building a production-quality Progressive Web App (PWA) for a solo lawyer in India to manage their practice. Read this entire brief before writing any code. Work in the phased order below — do not jump ahead to later phases before earlier ones are working end-to-end.

What this app is

A single-lawyer practice management tool covering: client & booking management, payments, a voice-to-text diary, court-form auto-fill, court-record lookup, and automated reminders. Primary user is a lawyer with no technical skills, using this mostly on their phone. Design and build for that person, not for a developer.

Hard constraints — do not deviate from these

This is a PWA, not a native app. No React Native, no Flutter, no Play Store/App Store submission. It must be installable via "Add to Home Screen" on Android and iOS.

Stack: Next.js (React) frontend, Supabase (Postgres + Auth + Storage) as the backend. No custom server to manage.

Payments are fully outsourced. Integrate Razorpay Payment Links + webhooks only. Never store, log, or touch raw card/UPI credentials anywhere in this codebase.

No secrets in code. All API keys (Supabase, Razorpay, WhatsApp BSP, Whisper) go in environment variables, never committed, never hardcoded in frontend bundles that ship to the browser.

Mobile-first. Design and test at phone width first; desktop is secondary.

Multilingual UI: English, Telugu, and Hindi, switchable from a settings toggle. Court-generated documents themselves stay in the legally required language (usually English) — only the app's own interface and WhatsApp reminder templates are multilingual.

Data privacy: this app holds privileged legal client data. Enable Supabase Row Level Security on every table from day one, scoped so only the authenticated lawyer's own data is ever queryable.

Feature specification

1. Client & booking management

Standard CRUD: client profile (name, contact, case reference, notes), and a bookings/appointments list tied to a client. Build this first — it's the foundation every other feature reads from.

2. Payments

Generate a Razorpay Payment Link per invoice/client.

Handle the Razorpay webhook to mark payments as received, and store the transaction against that client.

Build a per-client payment history view, and an overview view (e.g. total pending across all clients).

3. Digital voice diary

Record audio in-browser using the MediaRecorder API.

Send the audio to the Whisper API for transcription (fall back to the Web Speech API only if Whisper is unavailable).

Save the transcript, timestamp, and (optionally) the linked client against a diary entry. Store the raw audio file in Supabase Storage, not the database.

4. Court form auto-fill

Support a library of reusable form templates (start with the 3-5 most common ones — ask the user which, do not guess).

Map template fields to client-record fields, and generate a filled document (PDF or docx) on demand.

5. Court record lookup

Lookup by CNR (case number) only, triggered on-demand by the user — never a background sync.

Use an existing open-source library for eCourts data rather than writing a scraper from scratch.

Display case status, next hearing date, and case history in a simple read-only view.

6. Reminders

To the lawyer: an in-app notification/reminder list for upcoming bookings and hearing dates.

To the client: a WhatsApp message via the WhatsApp Business API (through a BSP such as AiSensy or Gupshup), using pre-approved utility templates, sent on a schedule (e.g. 24 hours before a booking).

Keep this as a separate, isolated module — it's the easiest feature to build wrong by tightly coupling it to bookings code.

Data model (starting point — refine with the user, don't treat as final)

`clients`: id, name, phone, email, case_reference, notes, created_at

`bookings`: id, client_id, datetime, purpose, status

`payments`: id, client_id, amount, razorpay_payment_id, status, paid_at

`diary_entries`: id, client_id (nullable), audio_url, transcript, created_at

`documents`: id, client_id, template_type, file_url, generated_at

`court_lookups`: id, client_id, cnr_number, last_result_json, checked_at

`reminders`: id, booking_id, channel (in_app / whatsapp), scheduled_for, sent_at

Build order — follow these phases in sequence

Scaffold the Next.js project with PWA support (manifest.json, service worker) and connect it to a fresh Supabase project with RLS enabled from the start.

Build client & booking management end-to-end. Get this fully working and installable on a phone before moving on.

Add Razorpay payment links, the webhook handler, and per-client payment history.

Build the voice diary (recording + Whisper transcription + storage).

Build court form auto-fill for a small starter set of templates.

Add court record lookup by CNR number.

Add the reminders module (in-app + WhatsApp).

Polish: offline caching strategy, install prompts, loading states, error states.

Do not start a phase until the previous one works on an actual phone, not just in a browser dev tools mobile emulator.

Design direction

This is a tool a working lawyer will use daily under time pressure — it should feel calm, trustworthy, and fast, not like a flashy consumer startup app. Avoid generic AI-generated-app defaults: no unnecessary gradient hero sections, no cookie-cutter rounded-card-with-soft-shadow layouts applied uniformly regardless of content, no all-caps section labels. Choose a restrained, professional color palette and a clean, highly readable typeface — legibility matters more than personality here, since the user will be scanning this quickly between client meetings. Keep navigation to a simple bottom tab bar or drawer — assume the user has never used a "productivity app" before, so label everything in plain language ("Add client," not "New record").

Build to this quality floor: fully responsive down to small phone widths, visible focus states for accessibility, adequate color contrast, and no unnecessary animation.

Non-functional requirements

Row Level Security on every Supabase table, scoped to the authenticated user.

Service worker caches the app shell so the app opens instantly even on a slow connection; show a clear offline state for anything requiring live data (payments, court lookups).

Every async action (saving a client, generating a payment link, transcribing audio) needs a visible loading state and a clear error message in plain language — the user cannot debug a silent failure.

Environment-based config so the same codebase can point at a staging Supabase project vs. the live one.

Explicit "do not"s

Do not integrate any native app store SDKs or submission tooling.

Do not build custom payment processing of any kind.

Do not store card numbers, CVVs, or any raw payment credentials anywhere.

Do not run court-record lookups as a background/scheduled job — on-demand only.

Do not hardcode API keys, tokens, or secrets anywhere in the repo.

Do not add features not listed in this brief without checking first — this is a fixed-scope v1.

Definition of done for each phase

Before marking any phase complete: test it by actually installing the app to a phone home screen and using that specific feature there, not just in a desktop browser. If you have browser-based verification available, use it to visually confirm the UI renders correctly at a phone viewport width before reporting the phase as finished.